

AI-103 · The Premium Exam Companion

Developing AI Apps and Agents on Azure — everything that's tested, nothing that isn't.

You're holding the full syllabus in one sitting. These notes condense all 8 sections of this course chapter (53 scripts + notebooks + labs) **plus** every official AI-103 syllabus topic and course-syllabus module — organized by the real exam domains, audited line-by-line against Microsoft's skills-measured outline.

How to read this guide

Pass	What to do
 First pass	Read top-to-bottom like a book — each domain opens with a one-breath summary so you always know where you are
 Second pass	Jump between the  tip boxes and  trap boxes — they mark exactly where exams win and candidates lose
 Exam week	Recap tables → flash facts → practice questions → done

The Exam at a Glance

Exam	AI-103 — Developing AI Apps and Agents on Azure
Certification	Microsoft Certified: Azure AI Apps and Agents Developer Associate
Skills measured as of	April 16, 2026 (AI-103 replaced AI-102, retired June 30, 2026)
Pass mark	700 / 1000
Question pool	Mostly GA features; commonly-used Preview features may appear
Companion course	AI-103T00 (4 days · 4 learning paths · 30 modules)

The blueprint — where the points live

#	Domain	Weight
1	Plan and manage an Azure AI solution	25–30%
2	Implement generative AI and agentic solutions	30–35%  the crown jewel
3	Implement computer vision solutions	10–15%
4	Implement text analysis solutions	10–15%
5	Implement information extraction solutions	10–15%

TIP

Weight hook: *"Plan a quarter, generate a third, see-say-extract a sixth each."* Domains 1 + 2 ≈ **60% of the exam** — agents and Foundry management are where you win.

The official course map (AI-103T00's four learning paths)

Learning path	Modules	Feeds domain
Develop generative AI apps in Azure (AI-3016)	Plan/prepare AI development → model catalog: select, deploy, evaluate → chat app with Foundry SDK + Responses API → apps that use tools → optimize (prompt eng / RAG / fine-tune) → responsible generative AI	1, 2
Develop AI agents on Azure (AI-3026)	Foundry Agent Service + VS Code → custom tools → MCP tools → Foundry IQ knowledge → Microsoft 365 / Work IQ integration → agent workflows → Microsoft Agent Framework → multi-agent orchestration → A2A protocol	2
Develop natural language solutions in Azure	Analyze text with Azure Language → text-analysis agent with Language MCP server → speech-capable generative AI app → speech-enabled apps (STT/TTS) → speech agent with Speech MCP server → Voice Live agent → translate text & speech	4
Extract insights from visual data on Azure (AI-3008)	Vision-enabled generative AI app → image generation → video generation (Sora 2) → analyze images with Content Understanding → multimodal analysis solution → Content Understanding client app/API → Document Intelligence → knowledge mining with Azure AI Search	3, 5

NOTE

Terminology decoder — Microsoft renamed things mid-flight, and the exam mixes both eras freely: "**Microsoft Foundry**" = Azure AI Foundry · "**Foundry Tools**" = the Azure AI services (Language, Speech, Vision, Translator, Document Intelligence, Content Understanding...)
· "**Foundry Models**" includes Azure OpenAI models.



DOMAIN 1 — Plan & Manage an Azure AI Solution

Weight: 25–30% · *The architect's domain*

In one breath: pick the right model and service for each job, stand it up securely without a single stored secret, watch it for drift and cost, and wrap the whole thing in responsible-AI guardrails. Every question here is really asking: *"would you trust this person to run AI in production?"*

1 · Choosing services and models

Need	Pick
General reasoning/chat/generation	LLM (GPT-family)
Low cost / low latency / edge	Small language model (SLM) (e.g. Phi family)
Code generation / completion	Code model (code-optimized deployments, e.g. Codex-class)
Images/audio/video input or output	Multimodal model
Purpose-built skill (OCR, translation, speech, PII...)	Foundry Tools (dedicated Azure AI service)
Generative tasks / grounding / vector search / agent workflows	Foundry Models / Foundry IQ + AI Search / Agent Service

- Model selection in the **model catalog**: compare by **benchmarks**, cost, context window, modality, region availability; evaluate manually (playground) and with automated evaluations before committing.
- Retrieval/indexing choice: keyword vs vector vs hybrid vs semantic (full story in Domain 5).
- Agent solutions additionally need: **memory** (threads/conversations), **tools** (function/OpenAPI/MCP), **knowledge** (Foundry IQ, AI Search, File Search).

🎯 **Scenario** → **service cheat sheet** (*Domain 1's favorite question type*)

The scenario says...	You answer
"chat / generate text or code"	Foundry Models (Azure OpenAI)
"ground answers in company data"	RAG: Foundry IQ / Azure AI Search + model
"agent that takes actions / calls tools"	Foundry Agent Service
"complex / multi-agent code-first orchestration"	Microsoft Agent Framework
"agents discover & talk to each other across systems"	A2A protocol
"agent inside Teams / Microsoft 365 Copilot"	M365 integration + Work IQ
"caption / describe / Q&A over an image"	Multimodal model (vision-enabled chat)
"create image from text"	Image generation model (gpt-image / DALL-E)
"create video from text"	Sora 2 video generation
"schema-driven extraction from docs/images/audio/video"	Content Understanding
"invoice / receipt / ID field extraction"	Document Intelligence prebuilt
"search over millions of docs, facets, filters"	Azure AI Search
"sentiment / NER / PII / summarize"	Azure Language (or LLM prompting — know the trade-off)
"translate text/documents"	Azure Translator (documents keep formatting)
"transcribe / synthesize / translate speech"	Azure Speech
"live voice conversation with an agent"	Voice Live API
"moderate content with thresholds"	Content Safety
"block jailbreaks / hidden prompt attacks"	Prompt Shields

2 · Setting up — infrastructure, deployment, CI/CD

- **Hub** → **project** structure (newer: Foundry-resource-based projects). Project = where you build; **connections** wire in external resources (search, storage, MCP servers) by name — the credential/endpoint indirection layer.
- Endpoint format: `https://<resource>.cognitiveservices.azure.com/` or `<resource>.services.ai.azure.com`; `AIProjectClient` is the project-level SDK entry; `get_openai_client()` bridges to the OpenAI-compatible surface.

Model deployment types — memorize this table

Type	Billing	Use when
Standard	Pay-per-token, regional	Default; variable workloads
Global Standard	Pay-per-token, global routing	Throughput/availability, no residency constraint
Global Batch	~50% cheaper, async (24 h)	Large offline jobs
Provisioned (PTU)	Reserved throughput units	Predictable latency, steady high volume
Serverless / MaaS	Pay-per-token, no infra	Catalog models (Llama, Mistral...)

★ IMPORTANT

CI/CD the exam way: provision via Bicep/ARM/Terraform · secrets from Key Vault or keyless auth · **automated evaluations as pipeline quality gates** before promoting a model/agent version · pin agent **versions** for reproducibility.

- Multi-service resource (one endpoint/key, many services) vs single-service (isolation + **F0 free tier**).
- Containers exist for edge/on-prem: run with `Eula=accept`, `Billing=<uri>`, `ApiKey=<key>` — billing telemetry goes to Azure, **your data doesn't**.

3 · Authentication & security — asked on EVERY exam

	API Key	Microsoft Entra ID (keyless)
Mechanism	<code>api-key</code> / <code>Ocp-Apim-Subscription-Key</code> header, <code>AzureKeyCredential</code>	OAuth bearer tokens via <code>DefaultAzureCredential</code>
Secret	Long-lived shared secret	Short-lived scoped tokens, no stored secret
Access control	Anyone with the key	RBAC role assignments
Verdict	Quick dev/test	Production standard

- Credential classes: `DefaultAzureCredential` (tries a chain — env vars → managed identity → CLI...), `AzureCliCredential` (local dev, fails fast), `ClientSecretCredential` (service principal), **managed identity** (Azure-hosted, no secret at all).
- Token scope for Foundry: `https://ai.azure.com/.default` via `get_bearer_token_provider`.
- **RBAC least privilege:** *Azure AI User / Developer* can **call** agents; managing needs broader roles; multi-resource apps (agent + search) need roles **per resource**.
- Two keys per resource → zero-downtime rotation; store keys in **Key Vault**; prefer **keyless (Entra ID)** everywhere it's supported.
- **Private networking:** private endpoints / VNet integration keep traffic off the public internet.
- Resilient pattern: *key if present, else* `DefaultAzureCredential` — same code local and in production.

💡 TIP

Memory hook: "*Key = quick, Entra = enterprise, keyless + private = production.*" When a question asks for the "most secure" design, the answer stacks all three: **managed identity + private endpoints + least-privilege RBAC**.

4 · Quotas, monitoring, and cost

- Azure OpenAI quota = **TPM (tokens per minute)** per deployment (RPM scales with it); handle 429s with retry/backoff; scale = raise quota, add deployments, or move to PTU.
- **Azure Monitor metrics** (calls, errors, latency, tokens) → alerts; **diagnostic settings** stream logs to **Log Analytics** (KQL) / Event Hub / Storage.
- AI-103 adds AI-specific monitoring: **model performance & drift**, **safety events** (filter triggers, jailbreak attempts), **grounding quality**, plus data-side health: **ingestion quality**, **search index health**, **relevance performance**.
- Cost levers: batch for bulk, SLMs where they suffice, cache/reuse, right-size deployments.

⚠ TRAP / WARNING

Trap radar: answers degrade a month after launch with the model unchanged? The exam wants you to look at the **retrieval layer** — index health, ingestion freshness, relevance metrics — not GPU utilization or quota.

5 · Responsible AI — a Domain-1 objective and a theme everywhere

The six Microsoft principles — memorize

Fairness · Reliability & safety · Privacy & security · Inclusiveness · Transparency · Accountability

💡 TIP

Mnemonic: F-R-P-I-T-A — "Fair Robots Protect Individuals Through Accountability."

The harm-prevention toolbox

Layer	What it does
Content filters	Per-deployment severity thresholds on the 4 harm categories; jailbreak + protected-material detections included
Content Safety service	Own resource; granular severity scores: Hate, SelfHarm, Sexual, Violence (🧠 <i>HSSV</i>), scale 0/2/4/6 (0–7 via <code>output_type</code>); custom blocklists (text-only) ; <code>analyze_text</code> / <code>analyze_image</code> are synchronous
Prompt Shields	Jailbreak (direct) + indirect prompt-injection attacks (instructions hidden in documents or images)
Groundedness detection	Flags claims unsupported by the provided sources
Evaluators	<code>azure-ai-evaluation</code> / portal: Completeness, Groundedness, Relevance, safety evaluations — run pre-deploy and continuously
Explanation tooling	Transparency in practice: surface <i>why</i> an answer was produced — citations/grounding sources, trace visualizations, evaluator score breakdowns — so outputs are explainable to users and auditors
Auditing	Trace logging, provenance metadata, approval workflows — who/what/why for every agent action
Agent governance	Oversight modes (autonomous ↔ human-approval), constraints, tool-access controls (<code>allowed_tools</code> , <code>require_approval</code>)

- The classic gate pattern: run Content Safety on user input, **block severity ≥ 4 (Medium+)** **before it reaches the model**.
- Trace **metadata** vs trace **content** are separable settings (`enable_content_recording`) — compliance/data-residency implication.
- The 4-stage responsible-GenAI process: **Identify** → **Measure** → **Mitigate** → **Operate** (map harms, evaluate, layer mitigations at model/safety-system/system-prompt/UX levels, then monitor in production).

DOMAIN 2 — Generative AI & Agentic Solutions

Weight: 30–35% · *The crown jewel — a third of your exam*

In one breath: speak the Responses API fluently, know *exactly* who executes which tool, build agents that remember, retrieve, and act — then wire them into knowledge platforms, Microsoft 365, workflows, and each other. If you master one domain deeply, make it this one.

6 · Clients, Responses API, and generation parameters

Client	Signature
Classic	<code>AzureOpenAI(azure_endpoint=..., api_key=..., api_version=...)</code>
v1 surface	<code>OpenAI(base_url="<resource>/openai/v1", api_key=...)</code> — no <code>api_version</code>
Via Foundry	<code>AIProjectClient(...).get_openai_client()</code>

- `model=` on Azure = your **deployment name**, not necessarily the model family name.
- **Responses API** = the unified, forward-path API (supersedes Assistants): multi-turn state, built-in tools, reasoning models, one call shape.
- `extra_body` = SDK escape hatch for Azure-specific fields (e.g. `agent_reference`).
- Parameters: `temperature` 0–2 (low = deterministic; prefer **structured outputs/JSON schema** over low temperature for parseable output), `max_output_tokens` (truncation is signalled), reasoning `effort` (quality vs latency + billed reasoning tokens), `tool_choice` (`auto` / `required` / `none` / named tool).
- `response.output_text` = text only; iterate `response.output` items for the full tool-call audit trail.

TIP

Spot-the-API in 2 seconds: `"type": "image_url"` inside `messages` → **Chat Completions**. `"type": "input_image"` inside `input` → **Responses API** (blocks: `input_text`, `input_image`, `input_file`). Code-reading questions love this.

7 · Tools & function calling — *the #1 scenario topic on the whole exam*

The golden distinction: WHO executes the tool?

Tool family	Examples	Executed by
Built-in / hosted	<code>web_search</code> , <code>code_interpreter</code> , <code>file_search</code> , <code>AzureAISearchTool</code>	The service , server-side
Custom function	<code>FunctionTool</code> , JSON-schema functions	YOUR code — model proposes name + args, then STOPS
OpenAPI tool	REST API + OpenAPI spec (+ <code>securitySchemes</code> for auth)	Service calls the API over HTTP
MCP tool	MCP server connection	Service → MCP server, guarded by <code>require_approval</code> + <code>allowed_tools</code>

TIP

Memory hook: *"Hosted tools run over there; function tools run right here."*

The function-calling loop (*identical across every LLM API*)

Request with tool schemas → model returns `function_call` (name + JSON args) **and stops** → **you execute** → resubmit `function_call_output` tagged with the matching `call_id` → final answer.

- JSON-Schema contract: `type`, `properties`, `required`, `additionalProperties: false`; zero-arg tools still need `{"type": "object", "properties": {}, "required": []}`; Python param names must match schema keys (`**arguments` unpacking).

- Foundry `FunctionTool` : declared schema (server) and implementation (client) are independent — **you keep them in sync**. Frameworks (`Runner.run` , `LangChain create_agent`) automate the loop.
- **Code Interpreter** = model-written Python in an isolated, ephemeral, service-managed container → deterministic math/data-analysis/charts.
- **MCP** (Model Context Protocol) = cross-vendor standard for tool discovery/invoke: transport → session → `initialize` → `list_tools` / `call_tool` ; tool results are **text content blocks** → return JSON *strings* for structured data.
- Azure Functions = the hosting layer for tool logic: HTTP triggers, OpenAPI backends, and native **MCP tool triggers** (auth levels: `ANONYMOUS` / `FUNCTION` / `ADMIN` ; key via `x-functions-key` header, enforced by the host **before your code runs**).

⚠ TRAP / WARNING

Trap radar: if the scenario shows a `function_call` item and asks "what happens next?" — the service does **nothing**. It's waiting for *your* code. Choosing "the service executes it automatically" is the most common wrong answer in this domain.

8 · Building agents with Foundry

- **Agent = model + instructions + tools + run loop.** `instructions` == system prompt. Define **role, goals, conversation-tracking approach, and tool schemas** up front.
- **Prompt agent** (`PromptAgentDefinition`) = stateless, per-request. **Hosted/persisted agent** + thread = server-side conversation memory. `conversations.create()` = server-side state on the OpenAI-compatible surface.
- `agent_reference` (via `extra_body`) invokes a persisted agent through a standard Responses call — its instructions, tools, `tool_choice` all apply; **pin a "version"** .
- Build/test in **portal playground or VS Code extension**; agents integrate **retrieval + function-calling + memory** in one definition.
- Grounding instruction *"answer only from sources; say you don't know otherwise"* = standard hallucination mitigation.

★ IMPORTANT

Two headline features AI-102 never had: - **Foundry IQ** — a **shared knowledge platform**: register knowledge sources once, many agents ground on them with consistent, **cited** responses. RAG-as-a-platform instead of per-agent wiring. - **Microsoft 365 integration** — publish Foundry agents to **Teams and Microsoft 365 Copilot**; **Work IQ** gives access to workplace data (mail, files, chats) with the user's permissions.

9 · Workflows, multi-agent, Agent Framework, A2A

- **Foundry workflows** (declarative YAML / visual): trigger → `InvokeAzureAgent` nodes → `ConditionGroup` routing → `EndConversation` ; capture agent output as **structured data** (`responseObject`) to route on it (e.g. intake-triage → knowledge agent vs ticket agent).
- **Microsoft Agent Framework** = open-source SDK, successor merging **Semantic Kernel + AutoGen** — code-first complex agents and multi-agent orchestration. Agent Service = managed hosting; Agent Framework = custom logic.
- Orchestration patterns: **sequential** (pipeline) · **concurrent** (fan-out/fan-in) · **group chat / debate-critic** · **handoff** (route to specialist) · Foundry `ConnectedAgentTool` .
- **A2A protocol** = Agent-to-Agent: standardized **discovery** (agent cards), direct communication, and coordinated task execution across **remote** agents.
- **Autonomous/semi-autonomous** workflows need **safeguards**: approval flow controls (human sign-off on risky actions), constraints, monitoring hooks; evaluate agent behavior and perform **error analysis** on traces.

💡 TIP

Protocol pairing, never confuse: MCP connects agents → tools. A2A connects agents → agents. Both are open, cross-vendor standards.

10 · RAG in applications

- Two styles: **bring-your-own retrieval** (your code queries the index, stuffs results into the prompt) vs **tool-based** (`AzureAISearchTool` / `FileSearchTool` /Foundry IQ retrieves server-side).
- Hybrid query mechanics: `search_text` + `VectorizableTextQuery` (integrated vectorization embeds the query for you); `k_nearest_neighbors` = vector-side candidates, `top` = final count; merged via **RRF (Reciprocal Rank Fusion)**.

- Chunking trade-off: small = precise but less context; large = more context, noisier; overlap preserves continuity.
- FAISS-in-a-notebook = learning stand-in; **the exam wants Azure AI Search / Foundry IQ** for production.

11 · Evaluation, optimization, observability

- **Evaluate** models AND apps: fabrication/hallucination detection, relevance, quality, safety — manual (playground) + automated (evaluators, LLM-as-judge with consistent rubrics).
- **Model reflection / self-critique loops**: model critiques and revises its own output; chain-of-thought evaluations — quality vs cost/latency trade-off (each critique = another LLM call).
- **Observability**: tracing (OpenTelemetry → Application Insights → Azure Monitor alerts), **token analytics**, **safety signals**, **latency breakdowns**; collect user feedback. Instrumentation wraps *around* the client (callbacks) — agent code unchanged.
- Orchestrate **multiple models** (router/planner: cheap SLM for easy turns, LLM for hard ones) and **hybrid LLM + rules engines** (deterministic rules for compliance-critical paths, LLM for open-ended ones).
- LangChain/LangGraph remain relevant as client-side orchestration: everything "agentic" there is **client-side**; LangGraph `StateGraph` = nodes + conditional edges; the 3-node `call_model ↔ tools` loop = canonical **ReAct**; no memory across `invoke()` runs unless you add a **checkpoint**.

★ IMPORTANT

★ **The optimization decision ladder** — probably the single most-tested judgment call on the exam: **1 Prompt engineering** — behavior/format; try first, cheapest, instant **2 RAG** — missing, fresh, or private **knowledge** **3 Fine-tuning** — consistent **style/format/domain vocabulary**; training + hosting cost

"Prompt for behavior, RAG for knowledge, fine-tune for style." Fine-tuning does **not** reliably add facts — that trap appears constantly. The three approaches **combine**.

DOMAIN 3 — Computer Vision Solutions

Weight: 10–15% · *Generation-first, understanding-second*

In one breath: AI-103's vision domain is about *creating* pixels (images, video, inpainting) and *understanding* them with multimodal models and Content Understanding — then policing both directions responsibly. The AI-102 classics are gone.

⚠️ TRAP / WARNING

Study-budget alert: Custom Vision, Face API, Video Indexer, and Spatial Analysis are **no longer in the outline**. Don't grind them (see the Legacy table near the end).

12 · Image & video generation

Operation	API	Key rules
Generate image	<code>images.generate()</code>	From text prompts (+ reference media); content filtering runs before generation
Prompt edit	<code>images.edit()</code> (no mask)	Less predictable — that's why masks exist
Masked edit = inpainting	<code>images.edit()</code> + mask	Only the masked region is regenerated
Generate video	Sora 2 in Foundry	Text prompt (+ reference media) → video; async job (submit-then-poll); editing workflows for generated video

⚠️ TRAP / WARNING

The inpainting trap (frequently mis-guessed): - Transparent ($\alpha = 0$) pixels get **REGENERATED**; opaque pixels are preserved pixel-for-pixel — the *opposite* of most people's intuition. - Mask must be a **PNG with the same dimensions** as the source; the source image for edits = **binary multipart file**, not URL/base64.

- Know the platform's generation/editing **controls**: size/aspect, quality, style, number of outputs, content-policy constraints.

13 · Multimodal understanding

- Vision-enabled chat: same Responses call with `input_image` blocks — hosted URL or base64 `data:` URI (base64 $\approx$ +33% payload, fully private). Not every deployment supports image input.
- Build: **concise or detailed captions** (single or multiple images), **visual question-answering grounded in the image evidence**, and **alt-text / extended descriptions aligned to accessibility guidelines** (convey *purpose*, not pixel-by-pixel detail).
- **Content Understanding for visuals**: extract visual characteristics, identify **objects/components/regions**, analyze **video segments**; know **single-task pipelines vs pro-mode** (pro mode = multi-step reasoning across multiple inputs/analyzers for complex extraction).

💡 TIP

OCR routing rule: text in **photos** → multimodal model / Vision READ · text in **documents** → Document Intelligence · **schema-driven multimodal** → Content Understanding.

14 · Responsible AI for multimodal content

- **Filter unsafe visual content**: image moderation via Content Safety `analyze_image` (HSSV categories) — only meaningful on multimodal deployments.
- **Indirect prompt injection via images** 🌟: attackers embed instructions as **text inside an image**; mitigate with Prompt Shields + treating OCR'd/image text as **untrusted data, never instructions**.
- **Visual policy enforcement**: watermarking generated media (provenance), flagging prohibited symbols, brand-usage rules, detecting inappropriate content before publishing.

DOMAIN 4 — Text Analysis Solutions

Weight: 10–15% · *Two roads to every answer*

In one breath: almost every text task has two implementations — a flexible LLM prompt and a deterministic dedicated service — and the exam's favorite move is making you choose. Then it turns up the volume: speech in, speech out, speech translated, speech *live*.

15 · LLM-based vs dedicated Language analysis — *know this trade-off cold*

	LLM prompting	Azure Language (Foundry Tools)
Categories	Arbitrary, invented on the spot (<code>ticket_id</code> , <code>sla_tier</code>), zero training	Fixed prebuilt taxonomy (Person, Location, Organization, DateTime, Quantity, Product, URL, Email, PhoneNumber...)
Output	Free-form unless schema-constrained (structured outputs)	Always the same fixed schema
Confidence	None	Numeric <code>confidence_score</code> per entity — threshold in production
Consistency	Can vary run-to-run	Deterministic, auditable

- One client, many ops: `TextAnalyticsClient` → `detect_language` (**ISO 639-1** out; detect first, then pass `language=` to other calls), `recognize_entities` , `recognize_pii_entities` (redaction), `analyze_sentiment` (sentence AND document level; `positive/neutral/negative/mixed` ; opinion mining = target + assessment pairs), `extract_key_phrases` .
- Summarization = **extractive** (picks sentences) vs **abstractive** (writes new text).
- LLM side: entity/topic/summary extraction with **structured JSON outputs** (Pydantic/JSON schema), sentiment/tone/safety/sensitive-content detection by prompt, **domain customization** (compliance summarization, domain extraction) via prompt engineering or fine-tuning.
- **Azure Language MCP server** 🌟: an agent calls Language operations (detect language, NER, **PII redaction**) as MCP tools — dedicated-service accuracy inside an agentic loop.
- Translation: **Azure Translator** (100+ languages; one call → many targets via the `targets` array; omit `from` = auto-detect; versioned REST + `Ocp-Apim-Subscription-Key` ; **document translation** = async batch between blob containers, **preserves formatting**) vs **LLM translation flows** (better tone/idiom, no formal guarantees).

⚠️ TRAP / WARNING

Trap radar: an LLM asked to "include confidence scores" will happily print numbers — but they're *generated text*, not statistics. When a scenario demands auditable confidence, the dedicated Language service is the only correct answer.

16 · Speech solutions

Mode	Method	Use for
Real-time / streaming	<code>start_continuous_recognition()</code> + <code>events(.recognizing = interim, .recognized = final)</code>	Live captioning, assistants
Single-shot	<code>recognize_once_async()</code>	Short commands, single turns
Batch	Submit file	Archives; adds per-word timestamps, speaker diarization , confidence

- `SpeechConfig` from `subscription + region` OR `subscription + endpoint` ; locales = `<language>-<region>` (`en-US`).
- **Neural voices** (`en-US-JennyNeural`) are the standard; plain text → `speak_text_async()` ; pronunciation/pauses/pitch/multi-voice → **SSML** via `speak_ssml_async()` .
- Check `.reason` vs `ResultReason` enums (`RecognizedSpeech` , `NoMatch` , `Canceled` , `SynthesizingAudioCompleted` , `TranslatedSpeech`).
- Speech translation = its own pair: `SpeechTranslationConfig` + `TranslationRecognizer` .
- **Custom speech models** for domain vocabulary/accents (quality = WER).

- New agentic speech 🌟 : **speech as an agent modality** (STT in → agent → TTS out) · **Azure Speech MCP server** (STT/TTS as MCP tools) · **Voice Live API/SDK** (real-time, low-latency full-duplex voice conversations with an agent — the "live voice agent" answer) · **multimodal reasoning over audio inputs** (audio-capable models reasoning directly over sound, not just transcripts).



TIP

Memory hook: *"Once for commands, continuous for conversations, batch for backlogs, Voice Live for live voice agents."*

DOMAIN 5 — Information Extraction Solutions

Weight: 10–15% · *From raw files to grounded answers*

In one breath: ingest anything (docs, images, audio, video), enrich it on the way in, index it three different ways, query it with precision — and hand the results to agents as clean, grounded context. The pipeline *is* the answer.

17 · Retrieval & grounding pipelines (Azure AI Search)

- **Ingest & index multimodal content:** documents, images, audio, video — enrich during indexing.
- Retrieval modes: **keyword** (BM25) · **vector** (k-NN over embeddings) · **hybrid** (both, RRF-merged) · **semantic ranker** (re-ranks top 50 BM25 results, adds captions + answers).
- Index field attributes: `key`, `searchable`, `filterable`, `sortable`, `facettable`, `retrievable`.
- Query syntax: simple vs **full Lucene** (`queryType=full` : wildcards `azur*`, fuzzy `~`, regex, boosting `^`); `$filter`, `$orderby`, `$select`, facets; `searchMode=any|all`.
- **Enrichment skillsets** (built-in + custom skills) run during indexing: e.g. **OCR** → **Merge** → **LanguageDetection** → **KeyPhrase** (this chapter's lab); custom skill = `WebApiSkill` → your **Azure Function**, strict `values / recordId` echo contract.
- **RAG ingestion flow:** docs → **OCR/layout** → chunk → embed → index; then **connect the pipeline to agents** as tools/knowledge (`AzureAISearchTool`, Foundry IQ knowledge sources).
- Plumbing: data source → indexer (schedule ≥ 5 min, change detection, enrichment cache) → skillset → index. Admin keys (manage) vs **query keys** (read-only clients).

⚠ TRAP / WARNING

Trap radar: `filterable` ≠ `searchable`. A field can be full-text searchable yet reject every `$filter` — and vice versa. 🧠 Attribute hook: *"Key Search Filters Sort Facets Retrieved."*

18 · Document extraction — Document Intelligence & Content Understanding

Document Intelligence prebuilt models

Model	Returns
<code>prebuilt-read</code>	OCR text only
<code>prebuilt-layout</code>	Structure: text, tables, selection marks — no semantic fields
<code>prebuilt-invoice</code>	Layout + semantic fields with confidence (VendorName, InvoiceId, InvoiceDate, DueDate, SubTotal, Tax, InvoiceTotal, line items)
receipt / ID document / business card	Match model to document type

- **Custom models:** **template** (fixed-layout forms) vs **neural** (varied docs), both need **5+ labeled samples** (Document Intelligence Studio); **composed model** = several customs behind one model ID with auto-classification.
- **LRO pattern** 🌟: `begin_analyze_document` / `begin_analyze_binary` return a **poller** (submit-then-poll) — pervasive across extraction SDKs.

Content Understanding

- **Multimodal pipelines:** OCR + layout analysis + field extraction in one; processes **documents, images, video, audio** under one API.
- Custom **analyzers** = `fieldSchema` on a `baseAnalyzerId` (e.g. `prebuilt-document`); `estimateFieldSourceAndConfidence` → value + confidence + source location per field.
- **markdown output** = clean, grounded, LLM-ready representation — feed it to agents/RAG; `fields` dict = programmatic access. Different analyzers → different result schemas; inspect before coding.

★ **IMPORTANT**

Pipeline golden rule: the extraction service **extracts** (reliable, schema-grounded); the LLM/agent **judges** (summarize, approve, flag). Keep the responsibilities separate — asking the LLM to also do raw field extraction is the wrong answer.

QUICK RECAP — Your Exam-Week Arsenal

19 · Legacy AI-102 topics — de-emphasized in AI-103

Expand only if you're curious — skim, don't grind

Was on AI-102	Status on AI-103
CLU (intents/entities/utterances), custom QA knowledge bases	Replaced by LLM/agent-based language understanding
Custom Vision (classification/object detection training)	Replaced by multimodal models + Content Understanding
Face API (verify/identify, PersonGroups)	Not in the outline
Video Indexer, Spatial Analysis	Replaced by Content Understanding video analysis
Knowledge store projections, prompt flow DAGs	Not emphasized; workflows/Foundry IQ carry the load

20 · ⚡ Rapid-fire tables

Which auth header?

Header / mechanism	Used by
<code>api-key</code>	Azure OpenAI
<code>0cp-Apim-Subscription-Key</code>	Foundry Tools REST (Language, Translator, Speech, Vision)
<code>x-functions-key</code> / <code>?code=</code>	Azure Functions
Bearer token (Entra ID, keyless)	Everything, in production

Sync or poll?

Call	Pattern
Content Safety <code>analyze_text</code> / <code>analyze_image</code>	Synchronous
<code>TextAnalyticsClient</code> operations	Synchronous
DocIntel / Content Understanding <code>begin_*</code>	LRO poller
Batch transcription, document translation, video generation	Submit-then-poll
Real-time speech, Voice Live	Streaming + events

Which number was that?

Number	Meaning
700	Pass mark
30–35%	GenAI + agentic domain weight (biggest)
0 / 2 / 4 / 6	Content Safety severities (0–7 finer via <code>output_type</code>)
≥ 4 gate	Block Medium+ before the model
0–2	<code>temperature</code> range
+33%	base64 image payload inflation
top 50	Results the semantic ranker re-ranks
5+	Labeled samples for a custom DocIntel model
5 min	Minimum indexer schedule interval
100+	Translator languages

Who executes it?

Thing	Executor
<code>web_search</code> , <code>code_interpreter</code> , <code>file_search</code>	Service (server-side)
<code>FunctionTool</code> / function calling	Your client code
OpenAPI tool	Service → your REST API
MCP tool	Service → MCP server (<code>require_approval</code> , <code>allowed_tools</code>)
LangChain/LangGraph tools	Your client process

Protocol pairing

Protocol	Connects
MCP	Agents → tools (discovery + invocation)
A2A	Agents → agents (discovery via agent cards, remote coordination)

Stateful vs stateless

Stateless	Stateful
<code>responses.create()</code> one-off	<code>conversations.create()</code> server-side state
<code>PromptAgentDefinition</code>	Hosted agent + thread
LangGraph <code>invoke()</code>	LangGraph + checkpointer

SDK class ↔ purpose

Class	Service
AzureOpenAI / OpenAI	Foundry Models (Azure OpenAI)
AIProjectClient	Foundry project
SearchClient , VectorizableTextQuery	AI Search
TextAnalyticsClient	Language
SpeechConfig + SpeechRecognizer / SpeechTranslationConfig + TranslationRecognizer	Speech
ContentSafetyClient	Content Safety
DocumentIntelligenceClient	Document Intelligence
ContentUnderstandingClient	Content Understanding
AzureKeyCredential / DefaultAzureCredential	Auth (key / keyless)

21 · 🚩 Flash facts — read 10 minutes before the exam

- AI-103 = "Developing AI Apps and Agents on Azure" → **Azure AI Apps and Agents Developer Associate**; agents + genAI = 30–35%.
- Entra ID / keyless > API key: short-lived tokens + RBAC; managed identity + private endpoints = "most secure" answer.
- `model=` on Azure = **deployment name**.
- `input_image` = Responses API; `image_url` = Chat Completions.
- Function tools: **model proposes, YOU execute**, resubmit with matching `call_id`.
- Built-in tools (web search, code interpreter, file search) run **server-side**.
- `tool_choice`: auto / required / none / specific.
- Prompt agent = stateless; hosted agent + thread/conversation = server-side memory.
- `extra_body` smuggles Azure fields (`agent_reference`) through the OpenAI SDK; pin agent versions.
- MCP connects agents to tools; A2A connects agents to agents** (agent cards for discovery).
- MCP safety levers: `require_approval` + `allowed_tools` ; MCP results = JSON **strings**.
- Foundry IQ** = shared knowledge platform: many agents, one grounded, cited knowledge layer.
- Work IQ + M365**: publish agents to Teams/Copilot with workplace-data access.
- Microsoft Agent Framework** = Semantic Kernel + AutoGen successor, code-first multi-agent; Agent Service = managed.
- Orchestration patterns: sequential, concurrent, group chat, handoff.
- Autonomous agents need approval flows, constraints, oversight modes, trace-based error analysis.
- Keyword = BM25; vector = k-NN; hybrid = RRF; semantic ranker re-ranks top 50.
- `k_nearest_neighbors` = vector candidates; `top` = final results.
- Decision ladder: **prompt-engineer** → **RAG** → **fine-tune** ("behavior → knowledge → style"); fine-tuning ≠ facts.
- Evaluators: Completeness, Groundedness, Relevance (+ safety evals); run in CI/CD as quality gates.
- Model reflection/self-critique = extra LLM calls = quality-vs-cost trade-off.
- Observability: tracing, **token analytics**, safety signals, latency breakdowns; content recording is a separate toggle.
- Hybrid orchestration: rules engines for deterministic paths, LLMs for open-ended ones; SLM router for cheap turns.
- Deployment types: Standard, Global Standard, Global **Batch** (~50% off), **PTU** (reserved), Serverless/MaaS; quota = **TPM**.
- Monitor AI-specifics: drift, safety events, grounding quality, index health, relevance.
- F-R-P-I-T-A: Fairness, Reliability, Privacy, Inclusiveness, Transparency, Accountability; process = Identify → Measure → Mitigate → Operate.
- Content Safety: **HSSV**, severities 0/2/4/6, sync calls, blocklists text-only; gate at ≥ 4.
- Prompt Shields: jailbreak (direct) + **indirect injection — including text embedded in images**.
- Groundedness detection flags claims unsupported by sources.
- Watermark/provenance + prohibited-symbol + brand rules = visual policy enforcement.
- Inpainting: **transparent = regenerated**, same-size PNG mask, binary multipart source.
- Sora 2** generates video from text/reference media in Foundry — async job.
- Alt-text: purpose-focused, accessibility-guideline-aligned; captions can be concise or detailed, single or multi-image.

- 34. Content Understanding: **single-task vs pro-mode** pipelines; analyzers = `fieldSchema` + `baseAnalyzerId` ; markdown output = LLM-ready.
- 35. OCR routing: photo → multimodal/Vision READ; document → DocIntel; multimodal schema → Content Understanding.
- 36. LLM = arbitrary categories, no confidence; Language = fixed schema + confidence scores — threshold them.
- 37. Sentiment = sentence + document level; opinion mining = target + assessment pairs.
- 38. Detect language first (ISO 639-1), then pass `language=` to other Language calls.
- 39. **Language MCP server** and **Speech MCP server** expose dedicated services as agent tools.
- 40. **Voice Live API** = real-time full-duplex voice agent conversations.
- 41. Translator `targets` array = many languages in one call; omit `from` = auto-detect; document translation = async blob-to-blob, keeps formatting.
- 42. SSML for pronunciation/pauses/pitch — via `speak_ssml_async()` , not a text-call parameter.
- 43. `TranslationRecognizer` → `ResultReason.TranslatedSpeech` ; custom speech quality = WER.
- 44. Index attributes: key, searchable, **filterable**, sortable, facetable, retrievable.
- 45. Full Lucene (`queryType=full`): wildcards, fuzzy `~` , boosting `^` .
- 46. Admin keys manage; **query keys** for read-only clients; indexer schedule ≥ 5 min.
- 47. Custom skill = WebApiSkill → Azure Function; echo the `recordId` s.
- 48. DocIntel: read = OCR, layout = structure, invoice = fields + confidence; template vs neural customs (5+ samples); composed = one ID, auto-classify.
- 49. `begin_*` poller = LRO across all extraction SDKs.
- 50. Extraction extracts, LLM judges — keep pipeline responsibilities separate.
- 51. Containers: `Eula` + `Billing` + `ApiKey` ; billing metadata to Azure, your data stays local.
- 52. Two keys per resource → zero-downtime rotation; secrets in Key Vault — or go keyless.
- 53. Multi-service resource = one key for many; single-service = isolation + F0 free tier.
- 54. CLU, custom QA, Custom Vision, Face, Video Indexer = **AI-102 legacy** — skim, don't grind.
- 55. Every "custom X" lifecycle: **Label** → **Train** → **Evaluate** → **Deploy** → **Consume**.

 The Final-Week Battle Plan

Day	Mission
-6 to -5	Full read of Domains 1 & 2 (your 60%) — dwell on every 🚩 trap box
-4	Domains 3, 4, 5 in one sitting — they share the extraction/pipeline mindset
-3	65 practice questions , timed. Score each domain separately
-2	Re-read only the sections behind your wrong answers; redo those questions
-1	Rapid-fire tables + flash facts, twice. Then stop — sleep beats cramming
Exam day	Flash facts over coffee ☕ → sit the exam → collect the badge

You've already **built most of this in the course**. The exam is just these patterns dressed up as scenarios — and now you recognize every one of them.

Go get it. 🚀

👉 *Test yourself:* [EXAM_PRACTICE_QUESTIONS.md](#) — 65 exam-style scenario questions with answers and explanations, weighted by the official blueprint.